DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Modelado y Simulación Guiados de Circuitos Secuenciales.**	**CÓDIGO:** **ANP3**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Libre. Ejercicios.	TIPO:	No presencial	DURACIÓN:	60 minutos

CALENDARIO:	Antes de la sesión presencial S7.	REQUISITOS:	Conocimientos adquiridos en Electrónica 2. Modelado VHDL de circuitos secuenciales. ModelSim. Testbenches.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

Esta actividad está dividida en tres partes. En la primera modelará en VHDL el comportamiento de un circuito secuencial sencillo. En la segunda simulará, mediante la herramienta *ModelSim*, el modelo anterior, para lo cual deberá escribir el correspondiente *testbench* para el circuito.

PARTE I. Modelo VHDL del Circuito

Deberá escribir el modelo de un circuito secuencial, llamado *reg_spe*, que es un registro serie-paralelo de 4 bits cuyas entradas y salidas son:

- Entrada de *reset* asíncrono *RST_an* activa a nivel bajo
- Entrada de reloj *CLK* activo en flanco de subida
- Entrada de habilitación *E* activa a nivel alto
- Entrada de datos en serie (un solo bit) *D*
- Salida de datos en paralelo (4 bits) *Q*

Con cada flanco activo de reloj, siempre que el registro se encuentre habilitado por *E*, los bits almacenados en *Q* se desplazarán hacia el lado de los bits de más peso y el bit presente en *D* se cargará en *Q(0)*.

Para ello (trabajando siempre dentro del directorio *ANP3* y respetando la estructura de directorios de la asignatura) deberá crear, en la herramienta *ModelSim*, un nuevo proyecto al que llamará *reg_spe*. Cree un nuevo fichero llamado *reg_spe.vhd* y realice en él la descripción del circuito.

Para el modelado del circuito puede serle de utilidad el *operador de concatenación* en VHDL «&». Este operador combina sus 2 operandos (de uno o varios bits) en un único resultado simplemente concatenando los bits de ambos operandos. Por ejemplo, el resultado de la operación `"01100" & "01"` es `"0110001"`. En el caso que nos ocupa el operador «&» puede ser útil para modelar la operación de desplazamiento, en la que los bits `Q(3 downto 0)` deben desplazarse a izquierdas cargando en *Q(0)* el valor de *D*, esto puede conseguirse mediante una asignación tal como:

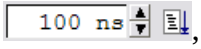
```
Q <= Q(2 downto 0) & D;
```

en la que *Q(3)* pasa a valer *Q(2)*, *Q(2)* pasa a valer *Q(1)*, *Q(1)* pasa a valer *Q(0)* y *Q(0)* pasa a valer *D*. Obviamente el valor anterior de *Q(3)* se pierde.

Con esto termina la primera parte de esta actividad.

PARTE II. Simulación

Para simular el circuito deberá generar un *testbench* para el mismo. Este *testbench* contendrá dos procesos: uno para generar el reloj y un segundo para generar el resto de estímulos.

En este caso se va a presentar un mecanismo que permitirá detener la simulación una vez se haya alcanzado un tiempo preestablecido, sin necesidad de tener que pulsar repetidamente el botón *Run*  , sino pulsando una sola vez el botón *Run -All*  (para lanzar la simulación) y esperando el fin de la simulación.

El proceso para la generación del reloj será ahora:

```
CLOCK : process
begin
  CLK <= '1';
  wait for TCLK / 2;
  CLK <= '0';
  wait for TCLK / 2;
  if now >= TEND then
    wait;
  end if;
end process CLOCK;
```

donde, como ve, se ha añadido un *if* que compara el objeto `now` con la constante `TEND`. En este caso la constante `TEND`, definida en la zona declaratoria de la arquitectura del *testbench* como:

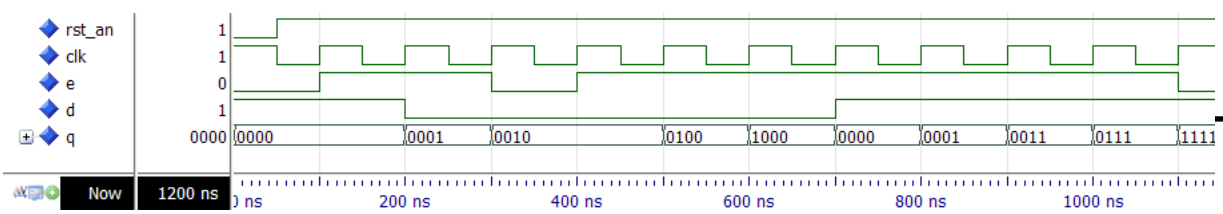
```
constant TEND : time := 12 * TCLK;
```

marca el tiempo final de la simulación (en este caso la simulación debe durar 12 ciclos de reloj). El objeto `now` es lo que en VHDL se conoce como una *función predefinida*. Basta saber que el objeto `now` indica, en cada momento, el tiempo (simulado) transcurrido desde el inicio de la simulación. Por tanto, la construcción:

```
if now >= TEND then
  wait;
end if;
```

compara el tiempo transcurrido en la simulación (`now`) con el tiempo final (`TEND`), cuando el primero sea mayor o igual que el segundo se ejecuta la sentencia `wait`, lo que hace que el proceso de generación del reloj se detenga, se deje de generar el reloj y, no habiendo más eventos que procesar, la simulación se detenga.

En este caso se pide que genere un *testbench* para el circuito anterior que aplique la secuencia de test siguiente.



Para realizar este *testbench* debe emplear construcciones del tipo *wait until* para ubicarse en aquellos instantes del tiempo en los que se producen los flancos de reloj, que son los instantes en los que deben modificarse las entradas del circuito. Como esqueleto de partida para la realización de este *testbench* puede emplear el siguiente código:

```
library ieee;
  use ieee.std_logic_1164.all;

entity reg_spe_tb is
end entity reg_spe_tb;

architecture TB of reg_spe_tb is
  constant TCLK : time := 100 ns;
  constant TEND : time := 12 * TCLK;
  signal RST_an : std_logic;
  signal CLK    : std_logic;
  signal E      : std_logic;
  signal D      : std_logic;
  signal Q      : std_logic_vector(3 downto 0);
begin
  DUV : entity work.reg_spe(RTL)
    port map(
      RST_an  => RST_an,
      CLK     => CLK,
      E       => E,
      D       => D,
      Q       => Q
    );

  CLOCK : process
  begin
    CLK <= '1';
    wait for TCLK / 2;
    CLK <= '0';
    wait for TCLK / 2;
    if now >= TEND then
      wait;
    end if;
  end process CLOCK;
```

```
  STM : process
  begin
    RST_an <= '0';
    E <= '0';
    D <= '1';
    wait until CLK'event and CLK = '1';  -- esperar un ciclo de reloj
    RST_an <= '1';
    -- reset asíncrono terminado
    wait until CLK'event and CLK = '1';  -- esperar el siguiente flanco de subida de CLK
      -- introduzca aquí el código restante

    wait;
  end process;
end architecture TB;
```

El fichero con el *testbench* debe llamarse *reg_spe_tb.vhd*. Guarde también el formato del visor de formas de onda en un fichero llamado *wave.do*.

Con esto termina la segunda parte de esta actividad.